

EXERCISE 1 — User Upcoming Events

Question

Show all upcoming events a user registered for in their own city.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
1 • SELECT u.full_name,
2       e.title,
3       e.city,
4       e.start_date
5 FROM Users u
6 JOIN Registrations r
7 ON u.user_id = r.user_id
8 JOIN Events e
9 ON r.event_id = e.event_id
10 WHERE e.status = 'upcoming'
11 AND u.city = e.city
12 ORDER BY e.start_date;
```

The result grid displays the following data:

full_name	title	city	start_date
Alice Johnson	Tech Innovators Meetup	New York	2025-06-10 10:00:00
Ethan Hunt	Frontend Development Bootcamp	Los Angeles	2025-07-01 10:00:00

EXERCISE 2 — Top Rated Events

Question

Find events with highest average rating.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
14 • SELECT e.title,
15       AVG(f.rating) AS average_rating,
16       COUNT(f.feedback_id) AS total_feedbacks
17 FROM Events e
18 JOIN Feedback f
19 ON e.event_id = f.event_id
20 GROUP BY e.event_id, e.title
21 HAVING COUNT(f.feedback_id) >= 1
22 ORDER BY average_rating DESC;
```

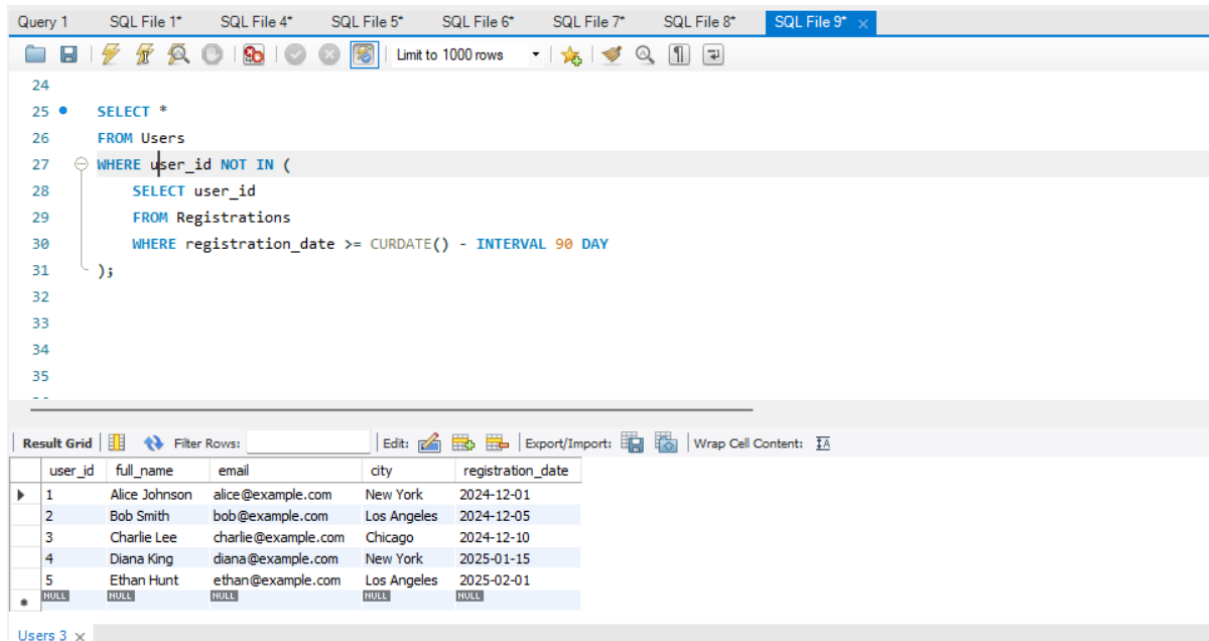
The result grid displays the following data:

title	average_rating	total_feedbacks
AI & ML Conference	4.5000	2
Tech Innovators Meetup	3.0000	1

EXERCISE 3 — Inactive Users

Question

Users who did not register in last 90 days.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
24
25 • SELECT *
26 FROM Users
27 WHERE user_id NOT IN (
28     SELECT user_id
29     FROM Registrations
30     WHERE registration_date >= CURDATE() - INTERVAL 90 DAY
31 );
32
33
34
35
```

The result grid displays the following data:

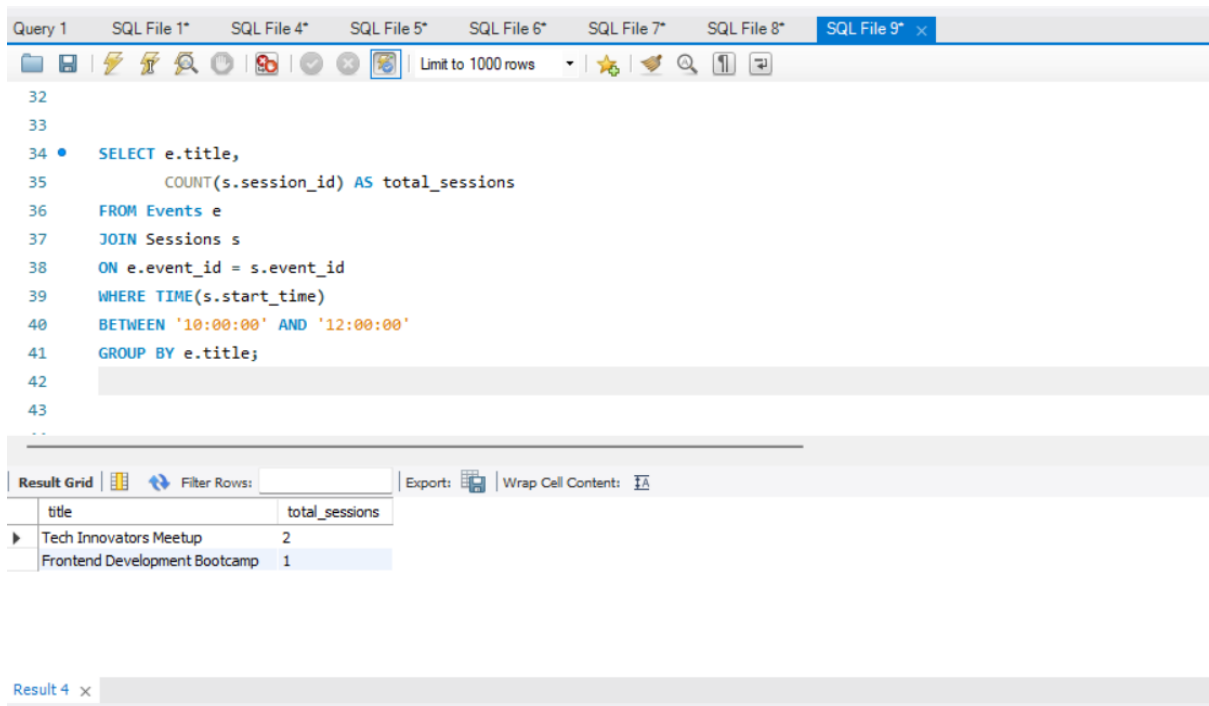
user_id	full_name	email	city	registration_date
1	Alice Johnson	alice@example.com	New York	2024-12-01
2	Bob Smith	bob@example.com	Los Angeles	2024-12-05
3	Charlie Lee	charlie@example.com	Chicago	2024-12-10
4	Diana King	diana@example.com	New York	2025-01-15
5	Ethan Hunt	ethan@example.com	Los Angeles	2025-02-01
NULL	NULL	NULL	NULL	NULL

Users 3 x

EXERCISE 4 — Peak Session Hours

Question

Count sessions between 10 AM and 12 PM.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
32
33
34 • SELECT e.title,
35     COUNT(s.session_id) AS total_sessions
36 FROM Events e
37 JOIN Sessions s
38 ON e.event_id = s.event_id
39 WHERE TIME(s.start_time)
40 BETWEEN '10:00:00' AND '12:00:00'
41 GROUP BY e.title;
42
43
```

The result grid displays the following data:

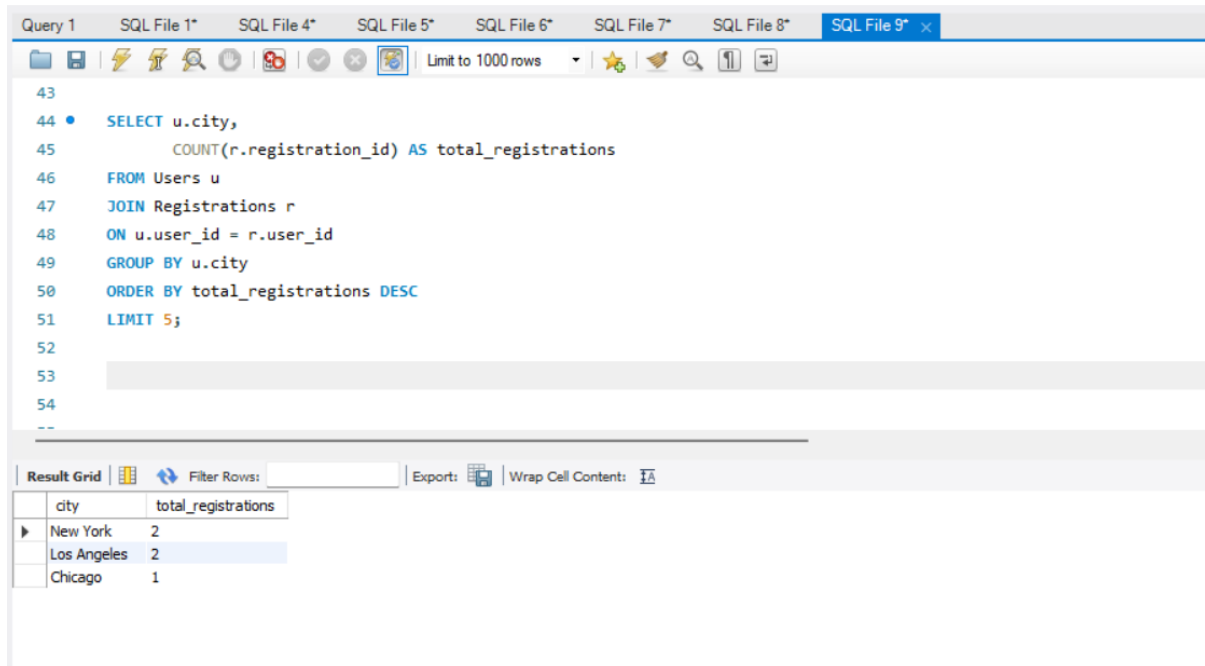
title	total_sessions
Tech Innovators Meetup	2
Frontend Development Bootcamp	1

Result 4 x

EXERCISE 5 — Most Active Cities

Question

Show top 5 cities with highest registrations.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
43
44 • SELECT u.city,
45       COUNT(r.registration_id) AS total_registrations
46 FROM Users u
47 JOIN Registrations r
48 ON u.user_id = r.user_id
49 GROUP BY u.city
50 ORDER BY total_registrations DESC
51 LIMIT 5;
52
53
54
55
```

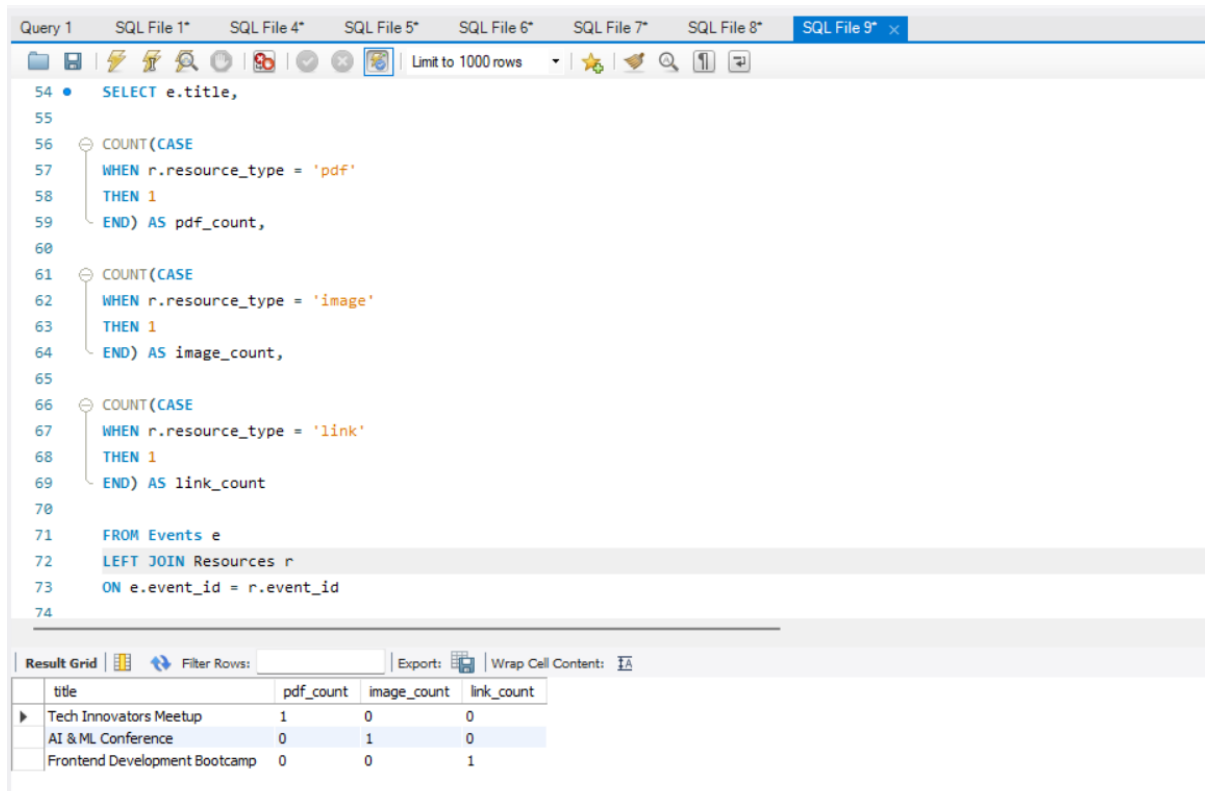
The result grid shows the following data:

city	total_registrations
New York	2
Los Angeles	2
Chicago	1

EXERCISE 6 — Event Resource Summary

Question

Count PDFs, images, and links for each event.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
54 • SELECT e.title,
55
56       COUNT(CASE
57         WHEN r.resource_type = 'pdf'
58         THEN 1
59       END) AS pdf_count,
60
61       COUNT(CASE
62         WHEN r.resource_type = 'image'
63         THEN 1
64       END) AS image_count,
65
66       COUNT(CASE
67         WHEN r.resource_type = 'link'
68         THEN 1
69       END) AS link_count
70
71 FROM Events e
72 LEFT JOIN Resources r
73 ON e.event_id = r.event_id
74
```

The result grid shows the following data:

title	pdf_count	image_count	link_count
Tech Innovators Meetup	1	0	0
AI & ML Conference	0	1	0
Frontend Development Bootcamp	0	0	1

EXERCISE 7 — Low Feedback Alerts

Question

Users who gave rating below 3.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
76
77 • SELECT u.full_name,
78         e.title,
79         f.rating,
80         f.comments
81 FROM Feedback f
82 JOIN Users u
83 ON f.user_id = u.user_id
84 JOIN Events e
85 ON f.event_id = e.event_id
86 WHERE f.rating < 3;
87
88
89
```

The result grid is empty, showing columns: full_name, title, rating, comments.

EXERCISE 8 — Sessions per Upcoming Event

Question

Show upcoming events with session count.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
90
91
92 • SELECT e.title,
93         COUNT(s.session_id) AS total_sessions
94 FROM Events e
95 LEFT JOIN Sessions s
96 ON e.event_id = s.event_id
97 WHERE e.status = 'upcoming'
98 GROUP BY e.title;
99
100
101
102
103
```

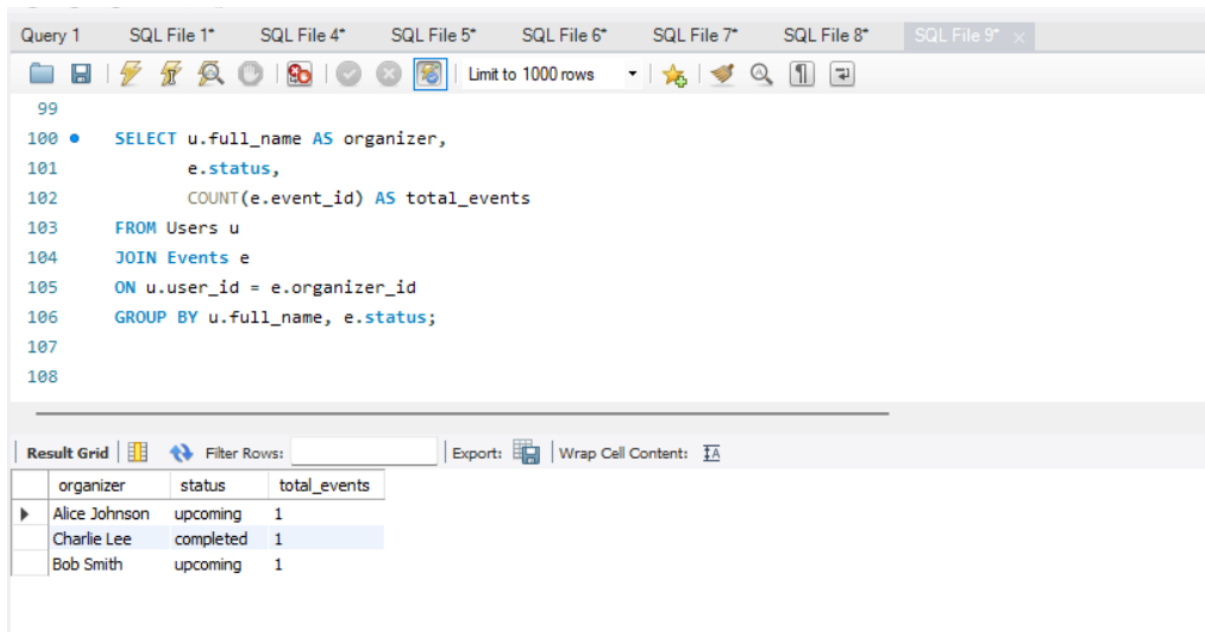
The result grid shows the following data:

title	total_sessions
Tech Innovators Meetup	2
Frontend Development Bootcamp	1

EXERCISE 9 — Organizer Event Summary

Question

Show organizer-wise event count and status.



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
99
100 • SELECT u.full_name AS organizer,
101         e.status,
102         COUNT(e.event_id) AS total_events
103 FROM Users u
104 JOIN Events e
105 ON u.user_id = e.organizer_id
106 GROUP BY u.full_name, e.status;
107
108
```

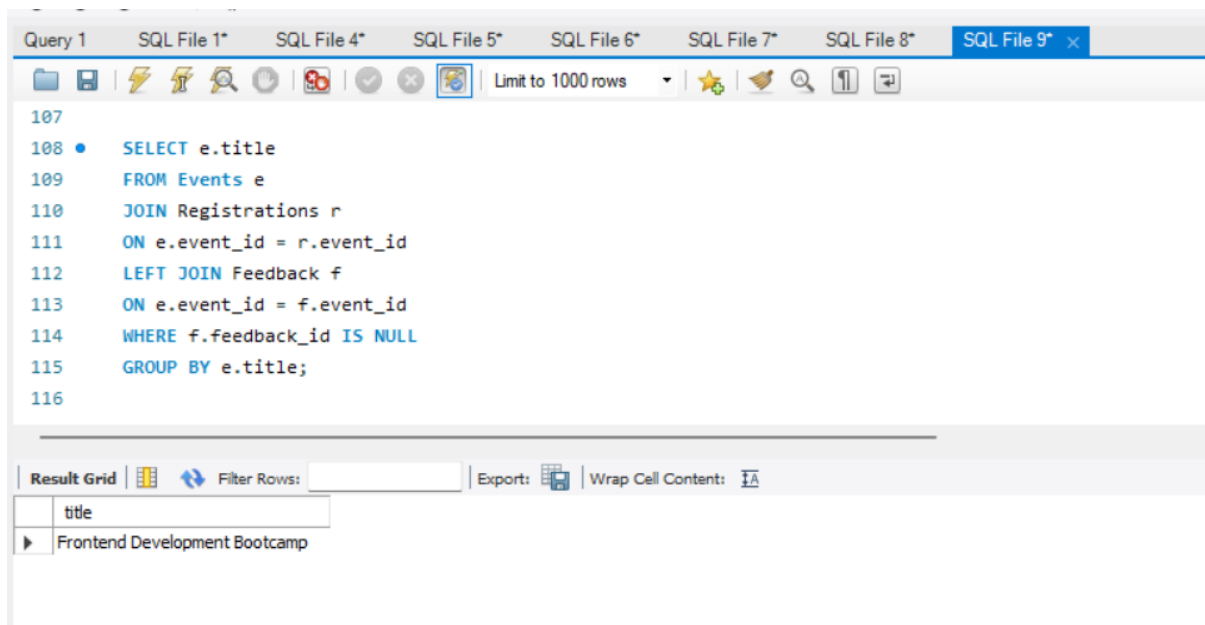
The results grid displays the following data:

organizer	status	total_events
Alice Johnson	upcoming	1
Charlie Lee	completed	1
Bob Smith	upcoming	1

EXERCISE 10 — Feedback Gap

Question

Find events with registrations but no feedback.



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
107
108 • SELECT e.title
109 FROM Events e
110 JOIN Registrations r
111 ON e.event_id = r.event_id
112 LEFT JOIN Feedback f
113 ON e.event_id = f.event_id
114 WHERE f.feedback_id IS NULL
115 GROUP BY e.title;
116
```

The results grid displays the following data:

title
Frontend Development Bootcamp

EXERCISE 11 — Daily New User Count

Question

Find number of users registered each day in last 7 days.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
117
118 • SELECT registration_date,
119         COUNT(user_id) AS new_users
120 FROM Users
121 WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
122 GROUP BY registration_date;
```

The result grid shows the following columns:

registration_date	new_users
-------------------	-----------

EXERCISE 12 — Event with Maximum Sessions

Question

Find event having highest number of sessions.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
123
124 • SELECT e.title,
125         COUNT(s.session_id) AS total_sessions
126 FROM Events e
127 JOIN Sessions s
128 ON e.event_id = s.event_id
129 GROUP BY e.title
130 HAVING COUNT(s.session_id) = (
131
132     SELECT MAX(session_count)
133     FROM (
134         SELECT COUNT(session_id) AS session_count
135         FROM Sessions
136         GROUP BY event_id
137     ) AS temp
138 );
```

The result grid shows the following columns:

title	total_sessions
Tech Innovators Meetup	2

EXERCISE 13 — Average Rating per City

Question

Find average feedback rating city-wise.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
139
140 • SELECT e.city,
141         AVG(f.rating) AS average_rating
142 FROM Events e
143 JOIN Feedback f
144 ON e.event_id = f.event_id
145 GROUP BY e.city;
146
147
148
149
```

The result grid displays the following data:

city	average_rating
New York	3.0000
Chicago	4.5000

EXERCISE 14 — Most Registered Events

Question

Find top 3 events with highest registrations.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
146
147 • SELECT e.title,
148         COUNT(r.registration_id) AS total_registrations
149 FROM Events e
150 JOIN Registrations r
151 ON e.event_id = r.event_id
152 GROUP BY e.title
153 ORDER BY total_registrations DESC
154 LIMIT 3;
155
156
```

The result grid displays the following data:

title	total_registrations
Tech Innovators Meetup	2
AI & ML Conference	2
Frontend Development Bootcamp	1

EXERCISE 15 — Event Session Time Conflict

Question

Find overlapping sessions inside same event.

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
155
156 • SELECT s1.event_id,
157           s1.title AS session1,
158           s2.title AS session2
159 FROM Sessions s1
160 JOIN Sessions s2
161 ON s1.event_id = s2.event_id
162 AND s1.session_id <> s2.session_id
163 AND s1.start_time < s2.end_time
164 AND s1.end_time > s2.start_time;
165
```

The result grid shows the following columns: event_id, session1, session2.

EXERCISE 16 — Unregistered Active Users

Question

Find users created in last 30 days but not registered for any event

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

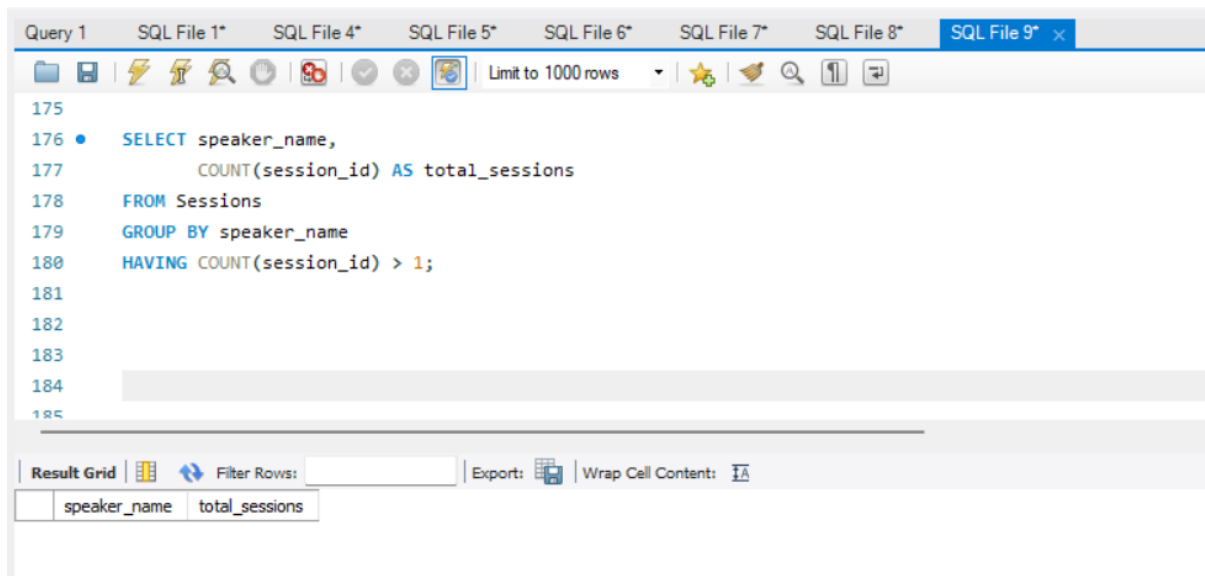
```
165
166
167 • SELECT *
168 FROM Users
169 WHERE registration_date >= CURDATE() - INTERVAL 30 DAY
170 AND user_id NOT IN (
171     SELECT user_id
172     FROM Registrations
173 );
174
175
```

The result grid shows the following columns: user_id, full_name, email, city, registration_date. The first row shows all NULL values.

EXERCISE 17 — Multi-Session Speakers

Question

Find speakers handling more than one session.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
175
176 • SELECT speaker_name,
177         COUNT(session_id) AS total_sessions
178 FROM Sessions
179 GROUP BY speaker_name
180 HAVING COUNT(session_id) > 1;
181
182
183
184
185
```

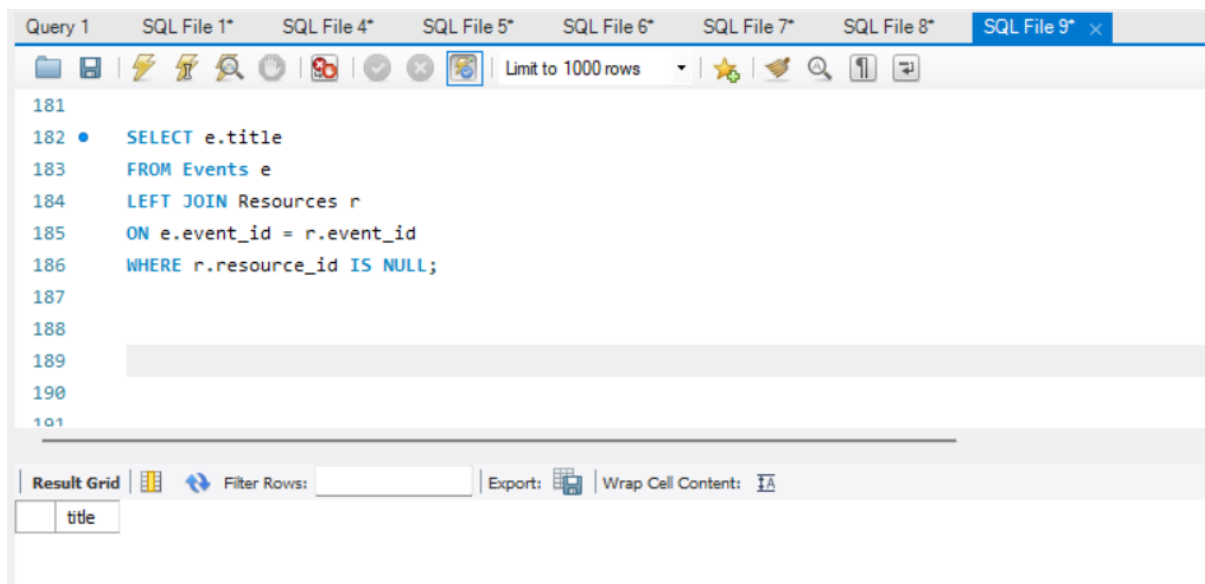
The result grid below the query editor shows the following columns:

speaker_name	total_sessions
--------------	----------------

EXERCISE 18 — Resource Availability Check

Question

Find events with NO resources uploaded.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
181
182 • SELECT e.title
183 FROM Events e
184 LEFT JOIN Resources r
185 ON e.event_id = r.event_id
186 WHERE r.resource_id IS NULL;
187
188
189
190
191
```

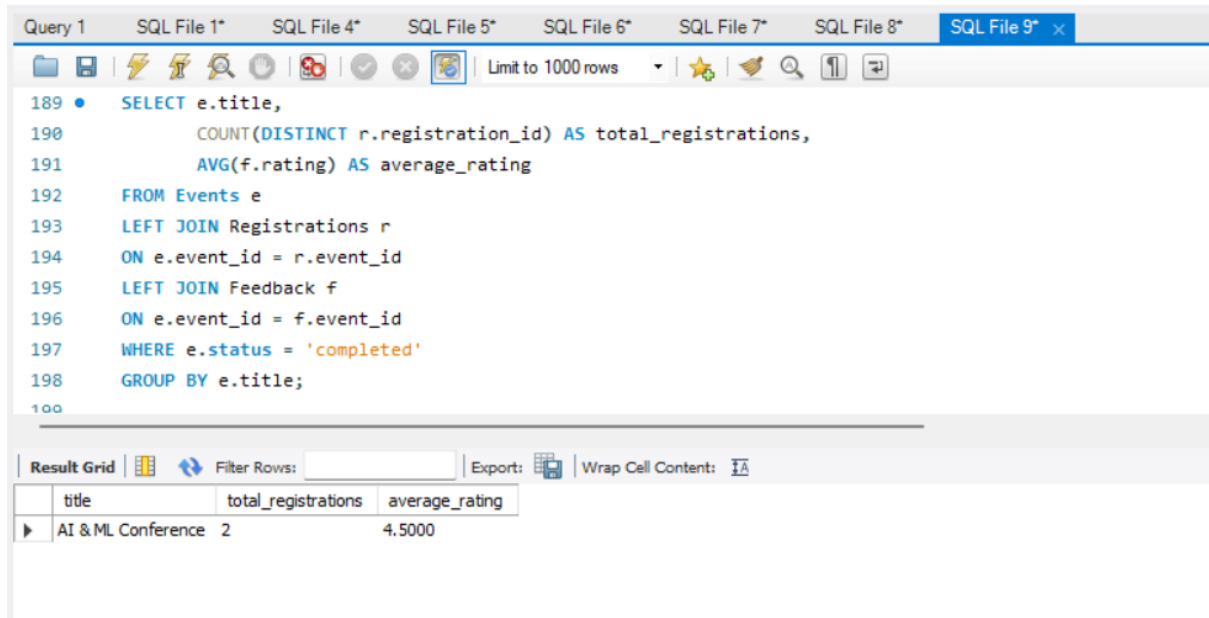
The result grid below the query editor shows the following columns:

title

EXERCISE 19 — Completed Events with Feedback Summary

Question

Show completed events with registrations and average feedback.



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
189 • SELECT e.title,  
190         COUNT(DISTINCT r.registration_id) AS total_registrations,  
191         AVG(f.rating) AS average_rating  
192 FROM Events e  
193 LEFT JOIN Registrations r  
194 ON e.event_id = r.event_id  
195 LEFT JOIN Feedback f  
196 ON e.event_id = f.event_id  
197 WHERE e.status = 'completed'  
198 GROUP BY e.title;  
199
```

The results grid shows the following data:

title	total_registrations	average_rating
AI & ML Conference	2	4.5000

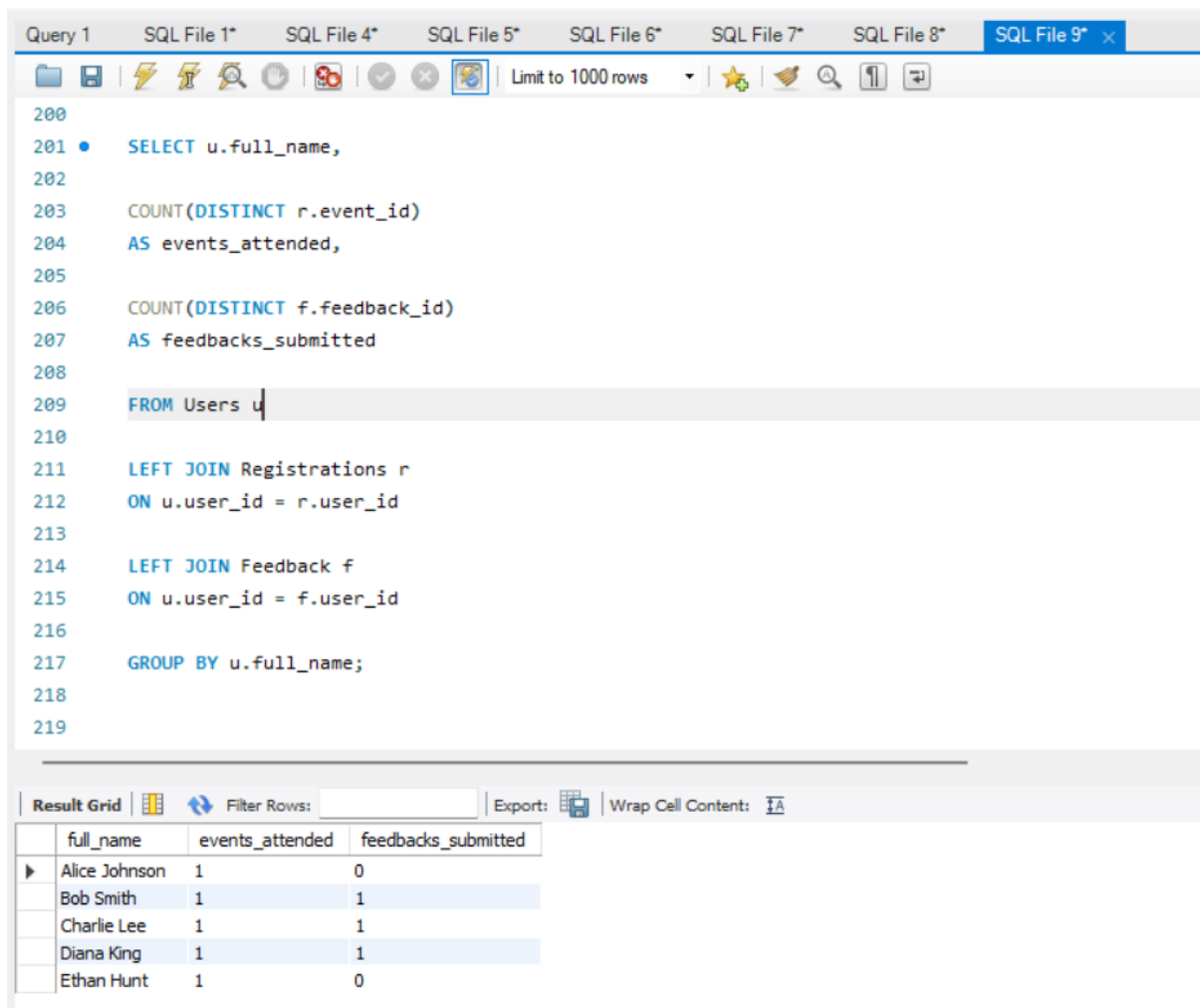
EXERCISE 20 — User Engagement Index

Question

Count:

- events attended
- feedback submitted

for each user.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
200
201 • SELECT u.full_name,
202
203     COUNT(DISTINCT r.event_id)
204     AS events_attended,
205
206     COUNT(DISTINCT f.feedback_id)
207     AS feedbacks_submitted
208
209 FROM Users u
210
211 LEFT JOIN Registrations r
212 ON u.user_id = r.user_id
213
214 LEFT JOIN Feedback f
215 ON u.user_id = f.user_id
216
217 GROUP BY u.full_name;
218
219
```

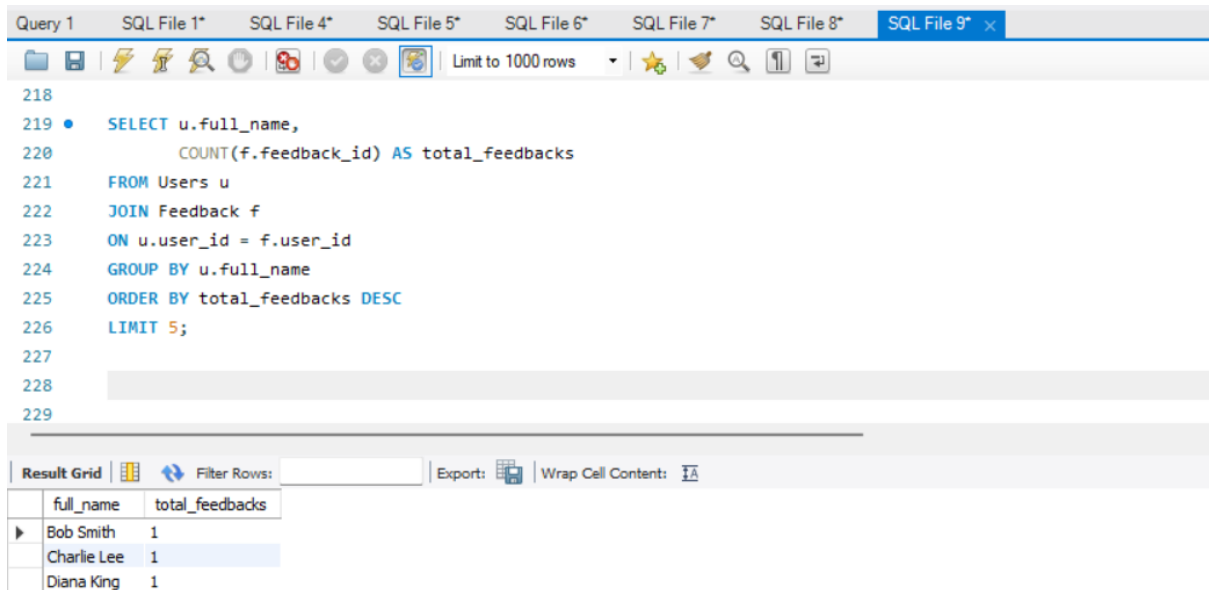
The result grid displays the following data:

	full_name	events_attended	feedbacks_submitted
▶	Alice Johnson	1	0
	Bob Smith	1	1
	Charlie Lee	1	1
	Diana King	1	1
	Ethan Hunt	1	0

EXERCISE 21 — Top Feedback Providers

Question

Find top 5 users who submitted most feedbacks.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
218
219 • SELECT u.full_name,
220         COUNT(f.feedback_id) AS total_feedbacks
221 FROM Users u
222 JOIN Feedback f
223 ON u.user_id = f.user_id
224 GROUP BY u.full_name
225 ORDER BY total_feedbacks DESC
226 LIMIT 5;
227
228
229
```

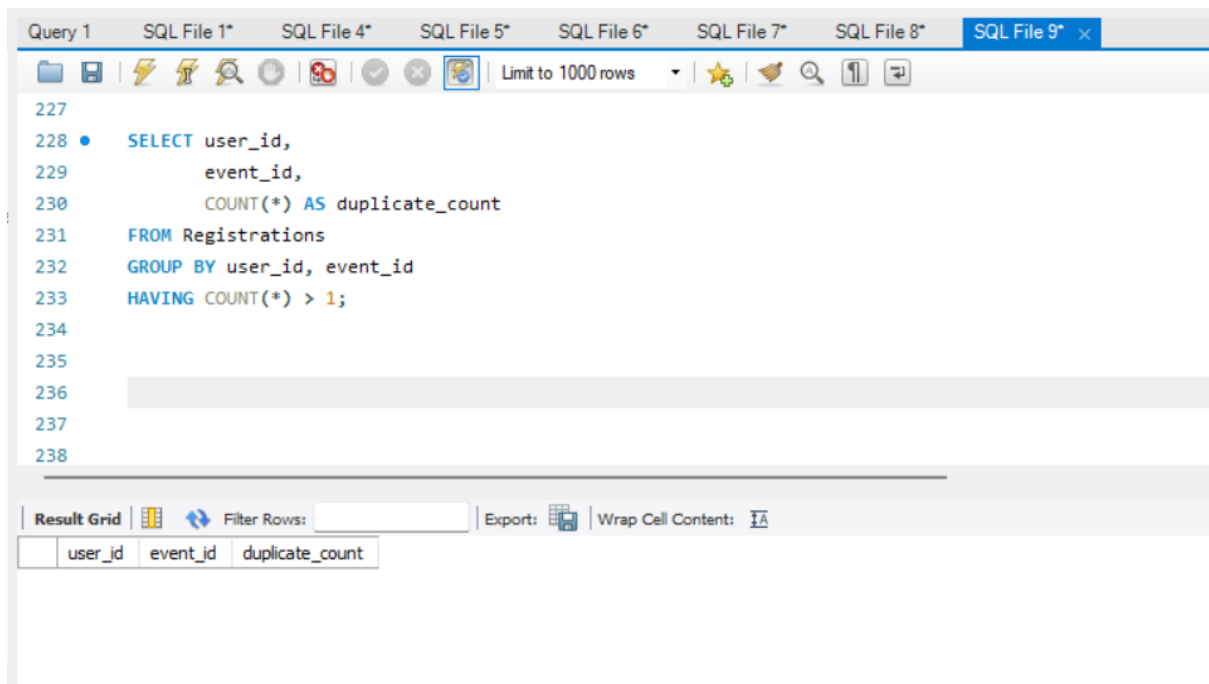
The result grid displays the following data:

full_name	total_feedbacks
Bob Smith	1
Charlie Lee	1
Diana King	1

EXERCISE 22 — Duplicate Registrations Check

Question

Find users registered more than once for same event.



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
227
228 • SELECT user_id,
229         event_id,
230         COUNT(*) AS duplicate_count
231 FROM Registrations
232 GROUP BY user_id, event_id
233 HAVING COUNT(*) > 1;
234
235
236
237
238
```

The result grid displays the following data:

user_id	event_id	duplicate_count
---------	----------	-----------------

EXERCISE 23 — Registration Trends

Question

Show month-wise registration count for past 12 months.

EXERCISE 24 — Average Session Duration per Event

Question

Find average session duration in minutes.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
244
245
246 • SELECT e.title,
247
248     AVG(
249     TIMESTAMPDIFF(
250     MINUTE,
251     s.start_time,
252     s.end_time
253     )
254     ) AS average_duration_minutes
255
256 FROM Events e
257 JOIN Sessions s
258 ON e.event_id = s.event_id
259
260 GROUP BY e.title;
261
262
```

The result grid at the bottom shows the following columns:

user_id	event_id	duplicate_count
---------	----------	-----------------

EXERCISE 24 — Average Session Duration per Event

Question

Find average session duration in minutes.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
244
245
246 • SELECT e.title,
247
248     AVG(
249     TIMESTAMPDIFF(
250     MINUTE,
251     s.start_time,
252     s.end_time
253     )
254     ) AS average_duration_minutes
255
256 FROM Events e
257 JOIN Sessions s
258 ON e.event_id = s.event_id
259
260 GROUP BY e.title;
261
262
```

The result grid shows the following data:

title	average_duration_minutes
Tech Innovators Meetup	67.5000
AI & ML Conference	90.0000
Frontend Development Bootcamp	120.0000

EXERCISE 25 — Events Without Sessions

Question

Find events having no sessions.

Query 1 SQL File 1* SQL File 4* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9*

Limit to 1000 rows

```

262
263
264
265
266 • SELECT e.title
267 FROM Events e
268 LEFT JOIN Sessions s
269 ON e.event_id = s.event_id
270 WHERE s.session_id IS NULL;
271
272
273
274

```

Result Grid Filter Rows: Export: Wrap Cell Content:

title

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	16:15:22	CREATE DATABASE event_management	1 row(s) affected	0.032 sec
2	16:16:08	USE event_management	0 row(s) affected	0.016 sec
3	16:17:16	CREATE TABLE Resources (resource_id INT PRIMARY KEY AUTO_INCREMENT, event_id INT, resource...	Error Code: 1824. Failed to open the referenced table 'events'	0.031 sec
4	16:17:22	CREATE TABLE Users (user_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT N...	0 row(s) affected	0.078 sec
5	16:17:29	CREATE TABLE Events (event_id INT PRIMARY KEY AUTO_INCREMENT, title VARCHAR(200) NOT NULL...	0 row(s) affected	0.078 sec
6	16:17:33	CREATE TABLE Sessions (session_id INT PRIMARY KEY AUTO_INCREMENT, event_id INT, title VARCH...	0 row(s) affected	0.078 sec
7	16:17:37	CREATE TABLE Registrations (registration_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT, event...	0 row(s) affected	0.078 sec
8	16:17:42	CREATE TABLE Feedback (feedback_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT, event_id I...	0 row(s) affected	0.094 sec
9	16:17:47	CREATE TABLE Resources (resource_id INT PRIMARY KEY AUTO_INCREMENT, event_id INT, resource...	0 row(s) affected	0.094 sec
10	16:19:04	INSERT INTO Users(full_name,email,city,registration_date) VALUES ('Alice Johnson','alice@example.com','New Yor...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.031 sec
11	16:19:11	INSERT INTO Events(title,description,city,start_date,end_date,status,organizer_id) VALUES ('Tech Innovators Meet...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
12	16:19:16	INSERT INTO Sessions(event_id,title,speaker_name,start_time,end_time) VALUES (1,'Opening Keynote','Dr. Tech',...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.031 sec
13	16:19:20	INSERT INTO Registrations(user_id,event_id,registration_date) VALUES (1,1,'2025-05-01'),(2,1,'2025-05-02'),(3,2,'...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.032 sec
14	16:19:25	INSERT INTO Feedback(user_id,event_id,rating,comments,feedback_date) VALUES (3,2,4,'Great insights!','2025-0...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec
15	16:19:31	INSERT INTO Resources(event_id,resource_type,resource_url,uploaded_at) VALUES (1,'pdf','https://portal.com/re...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
16	16:19:44	SELECT * FROM Users LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
17	16:21:12	SHOW DATABASES	9 row(s) returned	0.000 sec / 0.000 sec
18	16:21:41	USE event_management	0 row(s) affected	0.000 sec
19	16:21:52	SHOW TABLES	6 row(s) returned	0.000 sec / 0.000 sec
20	16:22:09	SELECT * FROM Users LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
21	16:22:23	SELECT * FROM Events LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
22	16:22:33	SELECT * FROM Sessions LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
23	16:22:47	SELECT u.full_name, e.title FROM Users u JOIN Registrations r ON u.user_id = r.user_id JOIN Events e ON r.event...	5 row(s) returned	0.016 sec / 0.000 sec
24	16:24:01	SELECT u.full_name, e.title, e.city, e.start_date FROM Users u JOIN Registrations r ON u.user_id = r.us...	2 row(s) returned	0.015 sec / 0.000 sec
25	16:29:28	SELECT e.title, AVG(r.rating) AS average_rating, COUNT(f.feedback_id) AS total_feedbacks FROM Events...	2 row(s) returned	0.000 sec / 0.000 sec
26	16:30:49	SELECT * FROM Users WHERE user_id NOT IN (SELECT user_id FROM Registrations WHERE registrati...	5 row(s) returned	0.016 sec / 0.000 sec
27	16:34:49	SELECT e.title, COUNT(s.session_id) AS total_sessions FROM Events e JOIN Sessions s ON e.event_id = s.ev...	2 row(s) returned	0.016 sec / 0.000 sec
28	16:36:34	SELECT r.registration_id AS total_registrations FROM Users u JOIN Registrations r ON u.user_id	3 row(s) returned	0.000 sec / 0.000 sec